ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO CUỐI KỲ

MÔN HỌC: ĐIỆN TOÁN ĐÁM MÂY

ĐỀ TÀI

SatiBot – Quản lý và tương tác với các tài nguyên AWS bằng ngôn ngữ tự nhiên

Giảng viên hướng dẫn:	TS. Hoàng Xuân Tùng
Nhóm thực hiện:	Nhóm 10
Sinh viên thực hiện:	Cao Anh Minh **(Trưởng nhóm)**
	Trịnh Thiên Quang
	Phạm Đức Hùng
	Nguyễn Đức Huy
Lớp môn học:	INT3319 1

Hà Nội, Tháng 11/2026

MỤC LỤC

1 TỔNG QUAN VỀ DỰ ÁN 1

1.1 Giới thiệu chung về tính năng dự án 1

1.2 Giới thiệu người dùng của dự án 1

1.3 Các dịch vụ AWS được hỗ trợ trong dự án 2

1.3.1 Nhóm kịch bản Amazon EC2 2

1.3.2 Nhóm kịch bản Amazon S3 2

1.3.3 Nhóm kịch bản AWS IAM 3

1.3.4 Nhóm kịch bản AWS Lambda 4

2 GIẢI PHÁP THỰC HIỆN 5

2.1 Kiến trúc tính năng của hệ thống 5

2.2 Kiến trúc triển khai của hệ thống 7

2.2.1 Sơ đồ kiến trúc triển khai hệ thống 8

2.3 Đặc tả chi tiết cần thiết trong giải pháp của dự án 9

2.3.1 Đặc tả cấu hình dịch vụ Bảo mật và Điều hướng 9

2.3.2 Giao diện và giao thức tương tác giữa các components 10

2.3.3 Cơ sở dữ liệu hệ thống 10

2.3.4 CI/CD 10

2.3.5 Phân quyền cho các hàm Lambda 12

2.3.6 Tích hợp LLM qua Gemini API 13

2.3.7 Endpoint tiếp nhận hội thoại và trích xuất ý định (POST /chat) 15

2.3.8 Endpoint xác nhận và thực thi tài nguyên (POST /confirm) . . 17

3 KẾT QUẢ THỰC HIỆN 19

3.1 Các bước triển khai hệ thống thực tế (Deployment Steps) 19

3.2 Nghiệm thu quy trình CI/CD và triển khai Canary không gián đoạn . . 20

3.3 Nghiệm thu Route 53 DNS, bảo mật AWS WAF và Origin Failover . . 21

3.4 Nghiệm thu các kịch bản quản trị AWS và đánh giá hiệu năng 22

3.4.1 Nghiệm thu sáu kịch bản Amazon S3 22

3.4.2 Nghiệm thu các kịch bản Amazon EC2 23

3.4.3 Nghiệm thu các kịch bản AWS IAM 23

3.4.4 Nghiệm thu kịch bản quản trị AWS Lambda qua giao diện ChatOps 23

3.4.5 Đánh giá hiệu năng vận hành thực tế và bài học tối ưu kỹ thuật 24

3.5 Vai trò và đóng góp của các thành viên 25

3.6 Liên kết sản phẩm, tài liệu bổ trợ và kết luận 25

1. TỔNG QUAN VỀ DỰ ÁN

1.1 Giới thiệu chung về tính năng dự án

Hiện nay, Amazon Web Services (AWS) là một trong những nền tảng điện toán đám mây phổ biến nhất phục vụ học tập, nghiên cứu và triển khai ứng dụng. Tuy nhiên, việc khởi tạo và quản trị tài nguyên thông qua giao diện web (AWS Management Console) đòi hỏi nhiều thao tác thủ công, điều hướng qua nhiều màn hình cấu hình phức tạp và rất dễ nhầm lẫn. Trong khi đó, việc sử dụng giao diện dòng lệnh (AWS CLI) hoặc viết tập lệnh tự động lại yêu cầu ghi nhớ chính xác cú pháp câu lệnh, cấu trúc tệp cấu hình JSON và các tham số kỹ thuật. Đối với sinh viên, lập trình viên mới tiếp cận điện toán đám mây hoặc người quản lý kỹ thuật, các rào cản này làm chậm quá trình vận hành, đồng thời tiềm ẩn nguy cơ cấu hình sai gây mất an toàn thông tin hoặc phát sinh chi phí ngoài ý muốn.

Nhằm giải quyết vấn đề trên, nhóm thực hiện đề tài xây dựng SatiBot – ứng dụng ChatOps hỗ trợ quản lý, cấu hình và giám sát tài nguyên AWS thông qua câu lệnh bằng ngôn ngữ tự nhiên tiếng Việt. Bằng cách kết hợp mô hình ngôn ngữ lớn (LLM) hỗ trợ cơ chế *Function Calling* với thư viện AWS SDK (Boto3), SatiBot hoạt động như một trợ lý hỗ trợ thao tác hạ tầng. Người dùng chỉ cần gửi yêu cầu bằng tiếng Việt qua giao diện chat; hệ thống sẽ phân tích ý định, kiểm tra tính hợp lệ và độ an toàn của câu lệnh, sau đó gọi các API tương ứng của AWS và trả về kết quả thực thi kèm theo thông tin chi tiết, cảnh báo rủi ro hoặc hướng dẫn xử lý tiếp theo.

1.2 Giới thiệu người dùng của dự án

Hệ thống SatiBot hướng tới 03 nhóm người dùng chính với nhu cầu sử dụng thực tế như sau:

- **Lập trình viên và kỹ sư vận hành (Developers / DevOps Engineers):** Cần khởi tạo nhanh môi trường thử nghiệm, cấu hình máy chủ, cập nhật mã nguồn hàm serverless hoặc xem log hệ thống trong quá trình phát triển ứng dụng. SatiBot giúp nhóm này thực hiện các tác vụ lặp đi lặp lại nhanh hơn ngay trên khung chat mà không cần chuyển đổi qua lại giữa nhiều trang trên AWS Console.
- **Sinh viên ngành Công nghệ thông tin / Điện toán đám mây:** Cần môi trường thực hành trực quan cho các môn học về mạng và hạ tầng đám mây. Hệ thống cung cấp cơ chế kiểm tra an toàn (guardrails), giúp sinh viên tránh các lỗi phổ biến như mở cổng mạng công khai nguy hiểm, quên tắt máy ảo hoặc chọn nhầm các gói cấu hình vượt quá phạm vi gói miễn phí (AWS Free Tier).
- **Quản lý kỹ thuật và chủ dự án (Tech Leads / Project Managers):** Cần theo dõi

nhanh tình trạng hoạt động của tài nguyên, kiểm tra phân quyền (IAM Audit) và kiểm soát chi phí sử dụng hàng tháng (FinOps) bằng ngôn ngữ tự nhiên mà không cần thành thạo cú pháp dòng lệnh AWS CLI.

1.3 Các dịch vụ AWS được hỗ trợ trong dự án

Để đáp ứng nhu cầu quản trị hạ tầng cơ bản, SatiBot hỗ trợ 04 dịch vụ chính của AWS gồm: Amazon EC2, Amazon S3, AWS IAM và AWS Lambda. Dưới đây là chi tiết về mục đích và quy trình tương tác thực tế của người dùng đối với từng nhóm tính năng của 04 dịch vụ AWS nòng cốt.

1.3.1 Nhóm kịch bản Amazon EC2

Dịch vụ máy chủ ảo (Amazon Elastic Compute Cloud - EC2) là hạ tầng phục vụ việc triển khai và chạy ứng dụng. SatiBot hỗ trợ 05 quy trình quản lý trạng thái và cấu hình máy ảo như sau:

1. **Khởi tạo máy chủ chuẩn hóa và xử lý trùng lặp tên:**
   Tạo nhanh máy ảo để thử nghiệm hoặc triển khai website mà không phải cấu hình thủ công nhiều bước, đồng thời xử lý trường hợp tên máy bị trùng.
2. **Kiểm soát chi phí và xử lý vượt hạn mức tài nguyên (Cost Guardrails & Quota):**
   Tránh việc chọn nhầm máy cấu hình cao gây phát sinh chi phí lớn, đồng thời hướng dẫn cách xử lý khi tài khoản chạm ngưỡng hạn mức tạo máy ảo (vCPU quota).
3. **Tra cứu và điều khiển vòng đời máy chủ (Bật / Tắt / Khởi động lại / Xóa):**
   Theo dõi danh sách máy ảo và điều khiển trạng thái bật/tắt kịp thời nhằm tránh lãng phí tài nguyên và kiểm soát chi phí vận hành.
4. **Giám sát hiệu năng và phát hiện máy chủ lãng phí (FinOps & CloudWatch):**
   Theo dõi mức độ sử dụng tài nguyên để tắt các máy nhàn rỗi nhằm tiết kiệm chi phí, hoặc nhận biết máy bị quá tải để kịp thời nâng cấp.
5. **Thiết lập tường lửa an toàn và hỗ trợ sự cố kết nối (SecOps):**
   Bảo vệ máy chủ khỏi nguy cơ bị quét dò mật khẩu từ Internet (brute-force attack) và hỗ trợ chẩn đoán nhanh nguyên nhân khi không kết nối SSH được vào máy ảo.

1.3.2 Nhóm kịch bản Amazon S3

Dịch vụ lưu trữ đối tượng (Amazon Simple Storage Service - S3) dùng để lưu trữ tập tin, bản sao lưu và dữ liệu tĩnh của hệ thống. SatiBot hỗ trợ 06 quy trình quản lý S3 như sau:

1. **Khởi tạo thùng chứa (S3 Bucket) an toàn và chuẩn hóa:**
   Tạo kho lưu trữ đối tượng được thiết lập ở chế độ riêng tư theo mặc định và tuân thủ quy tắc đặt tên của AWS.

2. **Quét và khóa rủi ro lộ dữ liệu ra Internet (Security Scan & Remediation):**
   Rà soát và ngăn chặn kịp thời tình trạng lộ dữ liệu do vô tình mở quyền truy cập công khai (Public Access) trên bucket.
3. **Xóa bucket S3 an toàn có kiểm soát (chống xóa nhầm dữ liệu):**
   Dọn dẹp bucket cũ mà không lo ngại nguy cơ xóa nhầm dữ liệu quan trọng đang lưu bên trong.
4. **Tối ưu chi phí lưu trữ lâu dài (FinOps Lifecycle Rules):**
   Giảm thiểu chi phí lưu trữ bằng cách tự động chuyển các tệp cũ, ít sử dụng sang các tầng lưu trữ giá rẻ hơn (S3 Standard-IA, S3 Glacier).
5. **Chia sẻ tệp tin an toàn có thời hạn (Presigned URL):**
   Cung cấp liên kết tải tệp tin an toàn cho người ngoài trong khoảng thời gian nhất định mà không cần mở public dữ liệu hay cấp quyền tài khoản AWS.
6. **Tra cứu dung lượng và tình trạng kho lưu trữ (Storage Analytics):**
   Nắm bắt nhanh số lượng tệp tin, dung lượng sử dụng thực tế và dự toán chi phí lưu trữ của từng bucket.

1.3.3 Nhóm kịch bản AWS IAM

Dịch vụ quản lý định danh và quyền truy cập (AWS Identity and Access Management - IAM) là trung tâm kiểm soát an ninh của hạ tầng AWS. SatiBot cung cấp 05 quy trình quản trị IAM an toàn, tuân thủ nguyên tắc đặc quyền tối thiểu (Least Privilege):

1. **Tra cứu vai trò (Role) và chính sách quyền hạn (Policy):**
   Xem nhanh danh sách hoặc kiểm tra chi tiết các quyền hạn đang được gán cho nhân sự hoặc dịch vụ để phục vụ công tác kiểm toán an ninh.
2. **Khởi tạo vai trò (Role) hoặc chính sách (Policy) mới theo nguyên tắc tối thiểu:**
   Tạo mới đối tượng phân quyền cho máy chủ, ứng dụng hoặc nhân sự mà không phải tự tay viết các đoạn mã JSON chính sách phức tạp và dễ nhầm lẫn.
3. **Cấp hoặc điều chỉnh quyền truy cập:**
   Bổ sung hoặc nới rộng quyền hạn cho nhân sự/dịch vụ khi phát sinh công việc mới trong dự án.
4. **Thu hồi quyền truy cập (Least Privilege Enforcement):**
   Gỡ bỏ các quyền hạn không còn sử dụng hoặc thu hồi các quyền nguy hiểm (như quyền AdministratorAccess cấp thừa cho tài khoản thực tập sinh) nhằm hạn chế rủi ro an toàn thông tin.
5. **Xóa đối tượng IAM an toàn:**
   Dọn dẹp dứt điểm các Role, Policy hoặc User cũ không còn sử dụng để tránh dư thừa tài nguyên và loại bỏ lỗ hổng bảo mật tiềm tàng.

1.3.4 Nhóm kịch bản AWS Lambda

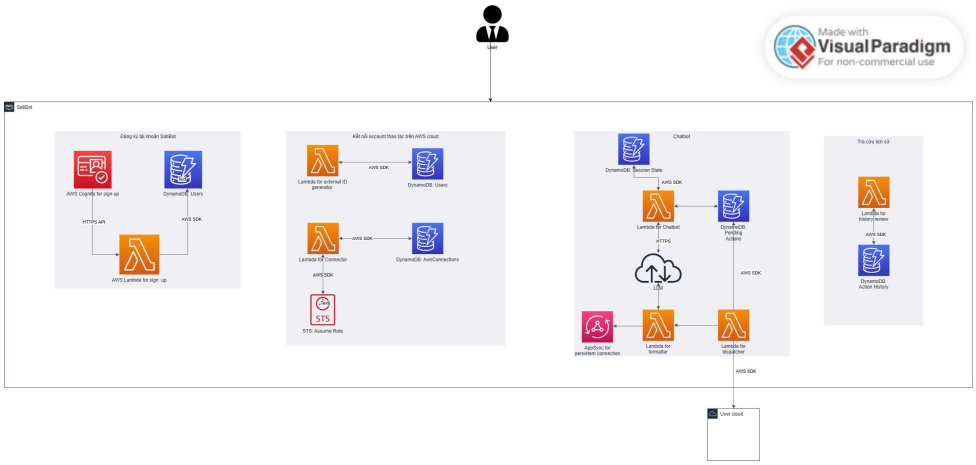
Dịch vụ điện toán không máy chủ (AWS Lambda) cho phép chạy mã nguồn theo sự kiện. SatiBot hỗ trợ 05 quy trình quản trị và kiểm thử hàm serverless trực tiếp:

1. **Tra cứu danh mục và kiểm tra cấu hình hàm (Inventory & Config Audit):**
   Nắm bắt nhanh toàn bộ các hàm Lambda đang có trong tài khoản cùng các thông số vận hành cốt lõi (ngôn ngữ runtime, bộ nhớ RAM, thời gian timeout) phục vụ công tác kiểm toán tài nguyên.
2. **Chạy thử nghiệm hàm trực tiếp qua Chat (Direct Invocation):**
   Kích hoạt và kiểm tra nhanh kết quả trả về của hàm với dữ liệu thử nghiệm ngay trên khung chat mà không cần đợi sự kiện bên ngoài hay cấu hình test event trên Console.
3. **Điều chỉnh cấu hình bộ nhớ và thời gian chờ của hàm Lambda:**
   Nâng cấp cấu hình tức thời cho các hàm đang chạy chậm hoặc có nguy cơ sập do thiếu bộ nhớ hay hết thời gian chờ mà không phải can thiệp mã nguồn hay mở Console.
4. **Cấp phát nhanh liên kết HTTP công khai (Function URL Generation):**
   Tạo nhanh một đường dẫn Endpoint HTTPS công khai (Function URL) để người dùng hoặc webhook bên ngoài có thể gọi trực tiếp hàm Lambda mà không mất công cấu hình API Gateway phức tạp.
5. **Tạm dừng hoạt động hàm khẩn cấp (Emergency Kill-Switch):**
   Vô hiệu hóa ngay lập tức một hàm đang bị lặp lỗi logic gây tiêu hao ngân sách hoặc cần ngắt kết nối gấp để bảo trì hệ thống mà không cần xóa bỏ mã nguồn của hàm.

2. GIẢI PHÁP THỰC HIỆN

Chương này trình bày kiến trúc mục tiêu và các hợp đồng kỹ thuật của giải pháp. Chương 3 đối chiếu thiết kế với các tài nguyên đã triển khai, kết quả kiểm thử và những phần chưa được nghiệm thu trên tài khoản AWS mới.

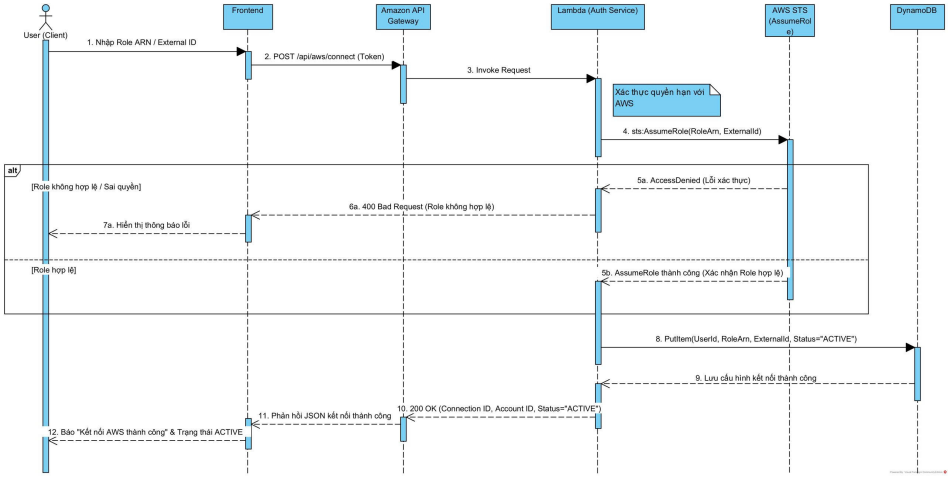
2.1 Kiến trúc tính năng của hệ thống



Hình 1. Sơ đồ kiến trúc tính năng và luồng xử lý nghiệp vụ của SatiBot

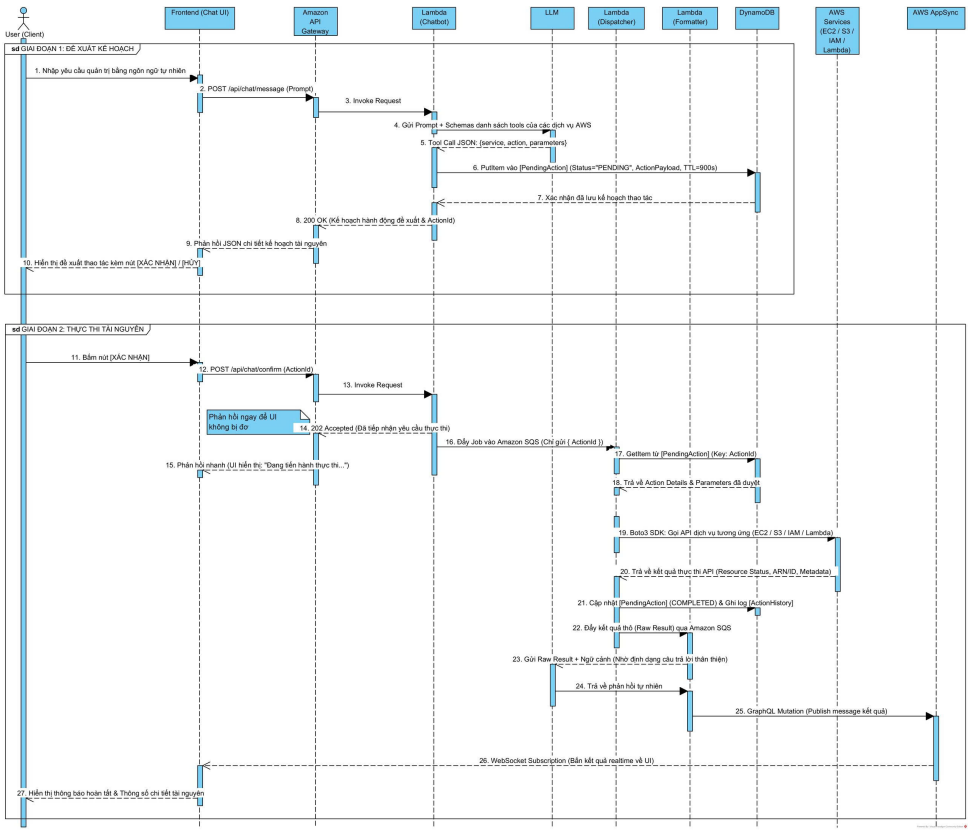
Các tính năng hệ thống cung cấp bao gồm:

1. **Đăng ký và xác thực người dùng:** Amazon Cognito xử lý đăng ký, đăng nhập và cấp JWT để xác thực các yêu cầu nghiệp vụ. Sau khi đăng ký, Lambda tạo hồ sơ người dùng trong bảng `Users` của DynamoDB.
2. **Kết nối tài khoản AWS:** SatiBot tạo `ExternalId`; người dùng dùng mã này và hướng dẫn của hệ thống để tạo IAM Role trong tài khoản AWS của mình, sau đó gửi `RoleArn` về SatiBot. Connector Lambda gọi STS `AssumeRole` để kiểm tra khả năng ủy quyền. Nếu thành công, hệ thống lưu thông tin kết nối và trạng thái trong bảng `AwsConnections`.



Hình 2. Sơ đồ tuần tự quy trình kết nối và xác thực tài khoản AWS mục tiêu

3. **Quản trị tài nguyên qua Chatbot:** Chatbot Lambda nhận câu lệnh, lấy ngữ cảnh hội thoại từ `SessionState` và gửi yêu cầu cùng Function Calling Schema đến mô hình LLM. Kết quả được lưu thành hành động chờ xác nhận trong `PendingAction`. Khi người dùng xác nhận, Chatbot Lambda chuyển ActionID vào SQS cho Dispatcher Lambda; Dispatcher Lambda dùng STS và Boto3 để gọi API của EC2, S3, IAM hoặc Lambda trên AWS của người dùng. Kết quả được lưu vào `ActionHistory` và gửi về giao diện qua AppSync.



Hình 3. Sơ đồ tuần tự quy trình điều phối và thực thi tác vụ ChatOps

4. **Tra cứu lịch sử thao tác:** History Lambda truy vấn bảng `ActionHistory` theo người

dùng và các điều kiện tra cứu, sau đó trả về thời gian, hành động, tài nguyên đích và trạng thái thực thi để hiển thị trên giao diện.

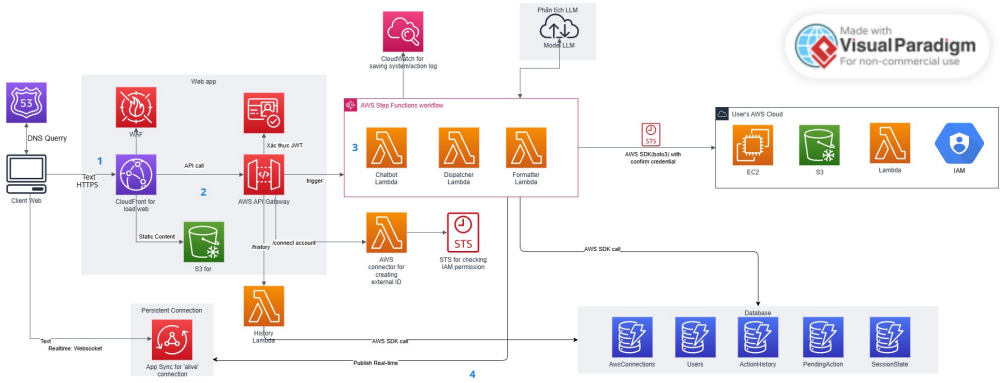
2.2 Kiến trúc triển khai của hệ thống

SatiBot sử dụng các dịch vụ AWS được quản lý để triển khai giao diện, xác thực, xử lý nghiệp vụ và lưu trữ dữ liệu.

Bảng 1. Các thành phần và dịch vụ trong kiến trúc triển khai hệ thống SatiBot

Thành phần / Dịch vụ	Vai trò trong kiến trúc
AWS WAF	Tường lửa ứng dụng web, gắn vào CloudFront để lọc request.
Amazon Route 53	Quản lý tên miền (DNS), điều hướng người dùng đến CloudFront.
React, Amazon S3 và CloudFront	Cung cấp giao diện ChatOps; S3 lưu các tệp tĩnh và CloudFront phân phối giao diện qua HTTPS.
Amazon Cognito	Quản lý đăng ký, đăng nhập và cấp JWT để xác thực người dùng.
AWS STS	Cấp quyền truy cập tạm thời (Temporary Credentials).
Amazon API Gateway	Tiếp nhận API từ frontend, xác thực JWT và định tuyến yêu cầu chat, xác nhận thao tác hoặc tra cứu lịch sử.
AWS Step Functions	Điều phối luồng thực thi sau khi người dùng xác nhận, gồm ChatBot, Dispatcher và Formatter Lambda.
AWS Lambda	Thực hiện các nghiệp vụ: Chatbot, Dispatcher, Formatter, Connector và History; Connector đảm nhiệm tạo ExternalId và kiểm tra kết nối AWS.
Mô hình LLM	Phân tích câu lệnh tiếng Việt và trả về ý định, tham số theo Function Calling Schema; không trực tiếp gọi API AWS.
AWS STS và Boto3	STS cấp thông tin xác thực tạm thời cho tài khoản đích; Boto3 dùng thông tin này để gọi API tài nguyên.
Amazon DynamoDB	Lưu hồ sơ người dùng, kết nối AWS, lịch sử hội thoại, hành động chờ xác nhận và lịch sử thao tác.
AWS AppSync	Phát kết quả qua GraphQL Subscriptions để giao diện cập nhật mà không cần polling liên tục.
Amazon CloudWatch	Lưu log thực thi và hỗ trợ theo dõi, chẩn đoán lỗi.
CI/CD Pipeline (GitHub, CodePipeline, CodeBuild)	Tự động hóa quy trình build, test và deploy.

2.2.1 Sơ đồ kiến trúc triển khai hệ thống



Hình 4. Sơ đồ kiến trúc triển khai tổng thể hệ thống SatiBot

Luồng hoạt động và giao tiếp giữa các thành phần gồm bốn bước:

1. **Tiếp nhận yêu cầu:** Người dùng truy cập ứng dụng web thông qua giao thức HTTPS. Yêu cầu được gửi đến AWS Route53 và trả về IP của CloudFront. Request sẽ đi qua AWS WAF để chặn các request xấu (như chặn SQL Injection, Cross-Site Scripting (XSS), hoặc giới hạn tần suất request để chống DDoS). Sau đó, Amazon CloudFront là nơi phân phối nội dung tĩnh (HTML, CSS, JS) từ Amazon S3. Các yêu cầu API động được CloudFront chuyển tiếp đến Amazon API Gateway. API Gateway sử dụng Amazon Cognito để xác thực JWT Token của người dùng
2. **Điều hướng yêu cầu:** Sau khi xác thực thành công, API Gateway kích hoạt dịch vụ tương ứng với yêu cầu của người dùng gồm Lambda for History (truy vấn lịch sử) và AWS core Step Function (Chatbot) .
3. **Thực hiện yêu cầu:** Bên trong Step Functions Workflow, các hàm AWS Lambda được triệu gọi tuần tự:
   - Chatbot Lambda gọi Model LLM để phân tích ý định của người dùng và trích xuất Tool Call. Trạng thái chờ xác nhận được lưu vào bảng `PendingAction` trong Amazon DynamoDB.
   - Sau khi người dùng xác nhận, Dispatcher Lambda gọi AWS STS để thực hiện `AssumeRole` lấy quyền truy cập tạm thời. Sau đó, nó sử dụng AWS SDK (`Boto3`) để thực thi các lệnh quản lý tài nguyên (như EC2, S3, IAM) trên tài khoản AWS cá nhân của người dùng.
   - Formatter Lambda gọi lại Model LLM để chuyển đổi kết quả kỹ thuật thô thành ngôn ngữ tự nhiên, đồng thời ghi log hành động vào bảng `ActionHistory` trong DynamoDB và log hệ thống vào CloudWatch.
4. **Ghi nhận và trả kết quả:** Formatter Lambda publish thông báo real-time qua AWS AppSync. AppSync duy trì kết nối WebSocket persistent và đẩy thông báo trực tiếp

về cho Client/Web

2.3 Đặc tả chi tiết cần thiết trong giải pháp của dự án

2.3.1 Đặc tả cấu hình dịch vụ Bảo mật và Điều hướng

1. Amazon Route 53

- **Tên miền & Subdomain:**
  - `satibot.id.vn` (Giao diện Web).
  - `api.satibot.id.vn` (API Gateway Endpoint).
- **Loại bản ghi:** Alias Record (trỏ tới CloudFront Distribution).
- **Chứng chỉ SSL/TLS:** AWS Certificate Manager (ACM).
  - Loại chứng chỉ: Wildcard (`*.satibot.id.vn`).
  - Region cấp phát: `us-east-1`.
- **Chính sách định tuyến:** Failover Routing.
- **Cấu hình bản ghi Primary:** Trỏ đến CloudFront của website chính, có S3 origin tại Singapore.
- **Cấu hình bản ghi Secondary:** Trỏ đến CloudFront Distribution (Region: `us-east-1`; Origin: S3 Bucket chứa tệp `error.html`).
- **Cấu hình Health Check (áp dụng cho bản ghi Primary):**
  - Protocol: HTTPS.
  - Path: `/health`.
  - Request Interval: 10 giây.
  - Failure Threshold: 3 lần.
- **TTL:** 60 giây.

2. AWS WAF (Web Application Firewall)

- **Vị trí tích hợp:** Gắn vào CloudFront Distribution.
- **Cấu hình Web ACL:**
  - **Managed Rule Groups:**

    `AWSManagedRulesCommonRuleSet`: Action = Block.

    `AWSManagedRulesSQLiRuleSet`: Action = Block.
  - **Rate-based Rule:**

    Threshold: 100 requests / 5 phút / 1 IP.

    Action: Block.

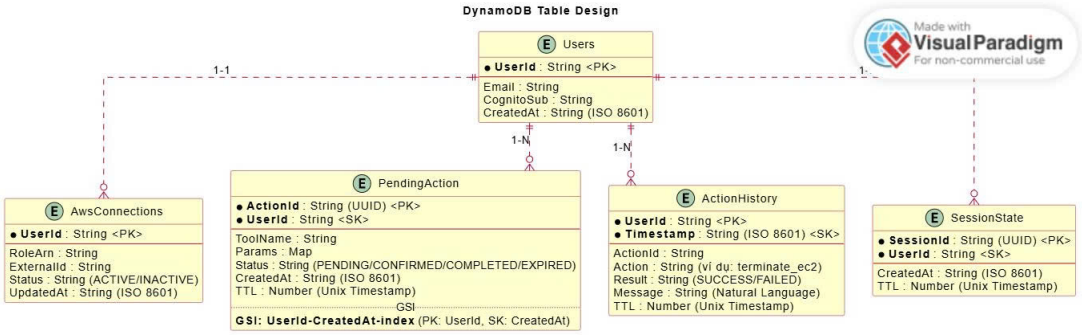
– **IP Sets:** Không cấu hình.

2.3.2 Giao diện và giao thức tương tác giữa các components

Bảng 2. Đặc tả giao diện và giao thức tương tác giữa các components

Nguồn → Đích	Giao thức	Xác thực	Dữ liệu trao đổi
Web Client → CloudFront → S3	HTTPS	TLS	Tệp giao diện tĩnh
Web Client → API Gateway	HTTPS REST	Cognito JWT	Câu lệnh, session ID, mã hành động
API Gateway → Chatbot Lambda	Lambda Proxy	IAM	Nội dung chat và thông tin phiên
API Gateway → Step Functions	StartExecution	IAM Role	User ID và mã hành động đã xác nhận
Chatbot Lambda ↔ LLM	HTTPS API	Theo nhà cung cấp LLM	Prompt, schema công cụ, tham số và kết quả phân tích
Step Functions → Lambda Tasks	AWS SDK Integration	Step Functions Execution Role	JSON đầu vào/đầu ra từng bước
Lambda ↔ DynamoDB	Boto3 SDK	Lambda Execution Role	GetItem, PutItem, Query, UpdateItem
Dispatcher Lambda → AWS STS	Boto3 / HTTPS	Lambda Execution Role	AssumeRole với RoleArn và ExternalId
Dispatcher → Tài khoản AWS đích	Boto3 / HTTPS	Credential tạm thời, SigV4	Yêu cầu API và kết quả thao tác AWS
Formatter Lambda → AppSync	GraphQL Mutation / HTTPS	IAM SigV4	Nội dung kết quả cần phát tới frontend
Web Client ↔ AppSync	GraphQL Subscription / WSS	Cognito User Pool	Sự kiện cập nhật trên khung chat

2.3.3 Cơ sở dữ liệu hệ thống



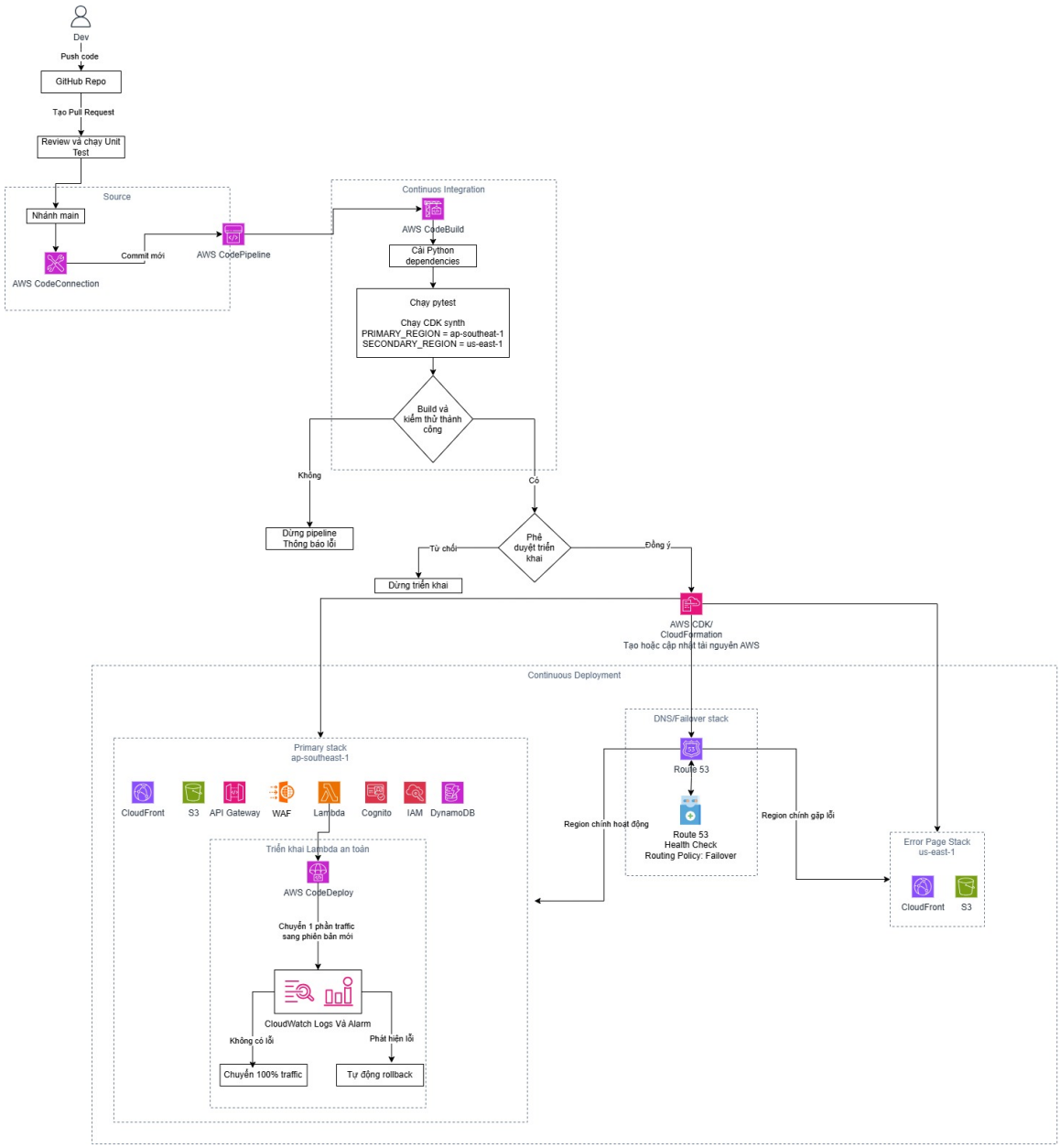
Hình 5. ERD cơ sở dữ liệu hệ thống

2.3.4 CI/CD

Theo thiết kế, quy trình CI/CD của SatiBot bắt đầu khi lập trình viên đẩy mã nguồn lên GitHub và tạo Pull Request. Mã nguồn được các thành viên trong nhóm xem xét, chạy kiểm thử đơn vị và chỉ được hợp nhất vào nhánh `main` khi đáp ứng yêu cầu. AWS CodeConnections theo dõi nhánh `main` và kích hoạt AWS CodePipeline khi xuất hiện commit mới. Trong giai đoạn tích hợp liên tục, AWS CodeBuild cài đặt Python cùng các thư viện cần thiết, chạy toàn bộ bộ kiểm thử bằng `pytest`, sau đó thực hiện `cdk synth` với hai biến cấu hình `PRIMARY_REGION=ap-southeast-1` và `SECONDARY_REGION=us-east-1`. Nếu quá trình build hoặc kiểm thử thất bại, pipeline dừng và phát thông báo lỗi; nếu thành công, quy trình chuyển sang bước phê duyệt thủ

công. Khi yêu cầu triển khai được chấp thuận, AWS CDK và AWS CloudFormation tiến hành tạo mới hoặc cập nhật các tài nguyên AWS theo cấu hình đã khai báo.

Theo thiết kế, hạ tầng triển khai liên tục được chia thành ba nhóm tài nguyên. Primary Stack tại `ap-southeast-1` chứa website chính và backend của SatiBot, bao gồm CloudFront, S3, API Gateway, WAF, Lambda, Cognito, IAM và DynamoDB. Các phiên bản Lambda mới được AWS CodeDeploy triển khai theo chiến lược canary; CloudWatch Logs và Alarm theo dõi trạng thái để chuyển toàn bộ lưu lượng sang phiên bản mới khi không có lỗi hoặc tự động rollback khi phát hiện sự cố. Error Page Stack tại `us-east-1` chỉ sử dụng S3 và CloudFront để cung cấp một trang thông báo lỗi độc lập, không cần đồng bộ dữ liệu ứng dụng với region chính. DNS/Failover Stack sử dụng Amazon Route 53 Health Check và chính sách Failover Routing để theo dõi hệ thống: khi region chính hoạt động bình thường, người dùng được điều hướng đến website tại `ap-southeast-1`; khi region này gặp sự cố, Route 53 chuyển kết quả phân giải DNS sang trang thông báo lỗi tại `us-east-1`.



Hình 6. Quy trình tích hợp, kiểm thử và triển khai liên tục của hệ thống SatiBot

2.3.5 Phân quyền cho các hàm Lambda

SatiBot phân quyền theo nhiệm vụ của từng Lambda. Mỗi hàm backend được gắn một IAM role để AWS xác định hàm được phép thực hiện thao tác nào và trên tài nguyên nào. Việc cấp quyền riêng giúp giới hạn tác động khi một hàm gặp lỗi hoặc xử lý sai yêu cầu.

Bảng 3 trình bày các quyền theo thiết kế cho năm Lambda backend và Lambda ứng dụng trong tài khoản người dùng. Các quyền đọc, ghi được giới hạn vào đúng bảng dữ liệu; quyền đọc khóa Gemini API chỉ được cấp cho những hàm sử dụng mô hình.

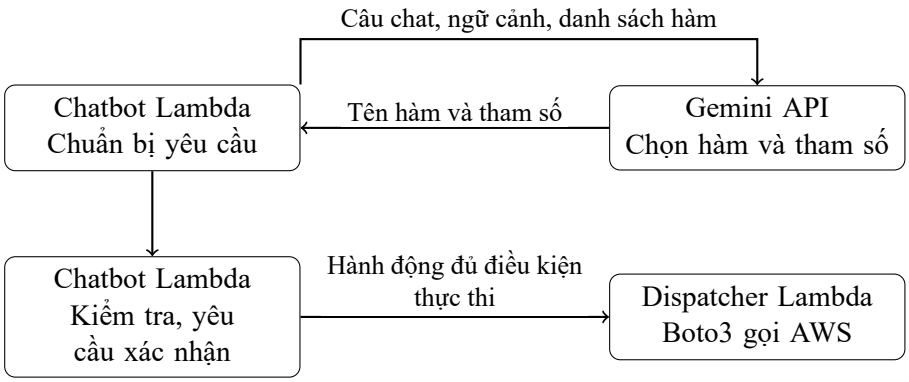
Bảng 3. Nhiệm vụ và quyền IAM của các Lambda

Lambda	Nhiệm vụ	IAM Action cần cấp và nơi sử dụng
Chatbot	Xử lý câu chat; lưu phiên và hành động chờ xác nhận.	`dynamodb:GetItem`: đọc SessionState, PendingAction và AwsConnections. `dynamodb:PutItem`, `dynamodb:UpdateItem`: ghi SessionState và PendingAction. `secretsmanager:GetSecretValue`: đọc khóa Gemini API.
Dispatcher	Thực thi hành động đã duyệt trên tài khoản AWS đích.	`dynamodb:GetItem`: đọc PendingAction và AwsConnections. `dynamodb:UpdateItem`: cập nhật PendingAction. `sts:AssumeRole`: nhận quyền tạm thời từ role trong tài khoản đích.
Formatter	Tạo lời đáp; lưu và gửi kết quả.	`dynamodb:PutItem`, `dynamodb:UpdateItem`: ghi ActionHistory. `appsync:GraphQL`: phát kết quả qua AppSync. `secretsmanager:GetSecretValue`: đọc khóa Gemini API.
Connector	Tạo ExternalId; lưu và kiểm tra kết nối AWS.	`dynamodb:GetItem`, `dynamodb:PutItem`, `dynamodb:UpdateItem`: quản lý AwsConnections. `sts:AssumeRole`: kiểm tra role người dùng cung cấp.
History	Liệt kê và xem chi tiết lịch sử.	`dynamodb:Query`: liệt kê bản ghi trong ActionHistory. `dynamodb:GetItem`: đọc một bản ghi trong ActionHistory.
Lambda của người dùng	Chạy mã ứng dụng trong tài khoản AWS đích.	Quyền phụ thuộc mã ứng dụng; ví dụ `s3:GetObject` khi hàm cần đọc tệp S3. Quyền quản lý hàm, như `lambda:InvokeFunction` hoặc `lambda:UpdateFunctionConfiguration`, nằm trong role đích mà Dispatcher đảm nhận.

Để thao tác trên tài khoản AWS của người dùng, Connector và Dispatcher sử dụng role do chủ tài khoản cho phép SatiBot truy cập. Connector kiểm tra kết nối; Dispatcher nhận quyền tạm thời qua STS để thực hiện các thao tác được role đó cho phép. Lambda ứng dụng của người dùng có quyền chạy riêng, tùy theo mã ứng dụng. Các Lambda ghi log được cấp thêm quyền ghi CloudWatch Logs để theo dõi lỗi.

2.3.6 Tích hợp LLM qua Gemini API

Hệ thống sử dụng mô hình Gemini của Google qua Gemini API. Tại Chatbot Lambda, mô hình chuyển câu chat tiếng Việt thành yêu cầu gọi hàm quản trị AWS; tại Formatter Lambda, mô hình diễn giải kết quả thực thi. Việc thao tác tài nguyên do Dispatcher thực hiện bằng Boto3 theo luồng ở Hình 3.



Hình 7. Tích hợp Gemini API với các Lambda xử lý yêu cầu

Backend gọi Gemini API bằng HTTP POST qua HTTPS, sử dụng endpoint Chat Completions do Google cung cấp. Nội dung request và response có định dạng JSON; khóa API được gửi trong header Authorization. Endpoint và mã model là cấu hình backend, model cần hỗ trợ Function Calling. Theo phương án triển khai, khóa API được lưu trong Secrets Manager và chỉ các Lambda cần gọi LLM được quyền đọc.

Bảng 4. Dữ liệu trao đổi giữa Chatbot Lambda và Gemini API

Chiều trao đổi	**Trường dữ liệu**	**Nội dung**
Gửi tới Gemini	`model`	Mã mô hình Gemini được cấu hình cho hệ thống.
	`messages`	Chỉ dẫn hệ thống, câu chat hiện tại và ngữ cảnh hội thoại cần thiết.
	`tools`	Danh sách hàm được phép gọi: tên, mô tả và JSON Schema quy định tham số.
	`tool_choice`	Giá trị `auto`: cho phép mô hình trả lời hoặc đề xuất gọi hàm theo yêu cầu.
Nhận từ Gemini	`content`	Nội dung trả lời hoặc câu hỏi bổ sung khi chưa đủ thông tin.
	`tool_calls`	Danh sách lời gọi hàm; mỗi lời gọi có tên hàm và các tham số.

Danh sách hàm được khai báo trước cho bốn module EC2, S3, IAM và Lambda. Mỗi khai báo mô tả một thao tác và các tham số bắt buộc, kiểu dữ liệu, giá trị cho phép. Khi nhận `tool_calls`, Chatbot kiểm tra tên hàm và tham số theo khai báo, rồi ánh xạ sang thao tác tương ứng của Dispatcher.

Formatter gửi kết quả AWS cho LLM để tạo lời đáp dễ đọc, không cần cung cấp danh sách hàm quản trị. Trạng thái thực thi, định danh tài nguyên và số liệu được lấy từ kết quả AWS; nội dung do LLM diễn giải không thay thế các dữ liệu này. Nếu bước diễn giải lỗi, Formatter trả kết quả theo mẫu có sẵn.

Mỗi lần gọi LLM có thời gian chờ và số lần thử lại giới hạn. Phản hồi sai cấu trúc không được chuyển sang thực thi. Ngữ cảnh hội thoại được giới hạn số lượt và dung lượng; credential AWS không được gửi tới mô hình. Lỗi kết nối hoặc lỗi tạo lời đáp

không làm chạy lại thao tác thay đổi tài nguyên đã thực hiện.

2.3.7 Endpoint tiếp nhận hội thoại và trích xuất ý định (POST /chat)

1. Thông tin chung:

- **Đường dẫn (Path):** `/chat` (hoặc `/api/chat` trên giao diện Web Client).
- **Phương thức HTTP:** `POST`.
- **Cơ chế xác thực:** Bearer JWT Token thông qua HTTP Header `Authorization: Bearer <Token>`. Amazon API Gateway sử dụng Cognito User Pool Authorizer để giải mã và kiểm tra tính hợp lệ của token trước khi kích hoạt hàm xử lý phía sau.
- **Thành phần tiếp nhận:** Amazon API Gateway → Chatbot Lambda (tích hợp mô hình ngôn ngữ lớn LLM với cơ chế Function Calling).

2. Dữ liệu trao đổi minh họa (Kịch bản khởi tạo máy chủ ảo EC2):

Mẫu JSON Request gửi từ Web Client khi người dùng gửi câu lệnh khởi tạo máy chủ EC2 bằng ngôn ngữ tự nhiên tiếng Việt:

```
{
  "message": "Tạo cho tôi một máy chủ EC2 Ubuntu có tên 'web-demo'",
  "history": [
    {
      "role": "user",
      "content": "Xin chào SatiBot"
    },
    {
      "role": "assistant",
      "content": "Xin chào! Tôi là SatiBot."
    }
  ],
  "region": "ap-southeast-1"
}
```

Mẫu JSON Response trả về từ hệ thống. Do thao tác `create_instance` là hành động làm thay đổi tài nguyên hạ tầng và phát sinh chi phí, hệ thống trả về mã trạng thái `CONFIRMATION_REQUIRED` kèm mã giao dịch `action_id` để yêu cầu xác nhận:

```
{
  "status": "CONFIRMATION_REQUIRED",
  "type": "action_confirmation",
```

```
  "action_id": "act_8f3a9e1b2c4d",
  "service": "ec2",
  "tool_name": "create_instance",
  "parameters": {
    "instance_name": "web-demo",
    "os_type": "ubuntu",
    "instance_type": "t3.micro"
  },
  "message": "Xác nhận tạo máy chủ EC2 web-demo?"
}
```

3. Bảng giải thích chi tiết các trường dữ liệu:

Bảng 5. Mô tả các trường trong Request Body của Endpoint `POST /chat`

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả ý nghĩa và nguồn gốc dữ liệu
`message`	String	Có	Nội dung câu lệnh bằng ngôn ngữ tự nhiên tiếng Việt của người dùng gửi từ giao diện.
`history`	Array[Object]	Không	Lịch sử hội thoại nhiều lượt gồm các cặp `role` và `content`, đồng bộ từ bảng `SessionState` trong DynamoDB.
`region`	String	Không	Khu vực AWS mục tiêu thực hiện tác vụ (mặc định: `ap-southeast-1`).
`session_id`	String	Không	Mã phiên hội thoại; giao diện dùng để ghép yêu cầu với lịch sử. Credential AWS không được gửi trực tiếp từ website.

Bảng 6. Mô tả các trường trong Response Body của Endpoint `POST /chat`

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả ý nghĩa và nguồn gốc dữ liệu
`status`	String	Có	Trạng thái phản hồi của hệ thống (`CONFIRMATION_REQUIRED` khi cần duyệt, hoặc `SUCCESS` khi chỉ truy vấn dữ liệu).
`type`	String	Có	Loại phản hồi định hướng hiển thị phía Client (`action_confirmation` hoặc `chat_reply`).
`action_id`	String	Có	Mã định danh duy nhất cho hành động chờ duyệt, được lưu làm Partition Key trong bảng `PendingAction` với thời gian TTL 15 phút.
`service`	String	Có	Dịch vụ AWS mục tiêu (ví dụ: `ec2`, `s3`, `lambda`, `iam`) do LLM Function Calling trích xuất.
`tool_name`	String	Có	Tên hành động kỹ thuật được trích xuất (ví dụ: `create_instance`, `manage_power`).
`parameters`	Object	Có	Cấu trúc tham số kỹ thuật chi tiết trích xuất từ câu lệnh tự nhiên của người dùng.
`message`	String	Có	Lời nhắn tiếng Việt thân thiện gửi người dùng để thông báo hoặc yêu cầu xác nhận an toàn.

2.3.8 Endpoint xác nhận và thực thi tài nguyên (POST /confirm)

1. Thông tin chung:

- **Đường dẫn (Path):** `/confirm` (hoặc gửi tới `/chat` với tham số `confirmed: true`).
- **Phương thức HTTP:** `POST`.
- **Cơ chế xác thực:** JWT Cognito trong header `Authorization`.
- **Thành phần tiếp nhận:** Amazon API Gateway → Chatbot Lambda / Step Functions → Dispatcher Lambda → EC2 Service (`boto3.client('ec2')`) → Amazon DynamoDB (`ActionHistory`).

2. Dữ liệu trao đổi minh họa (Kịch bản người dùng chấp thuận tạo máy ảo):

Mẫu JSON Request gửi từ Web Client khi người dùng nhấn nút "Xác nhận thực thi" trên giao diện:

```
{
  "confirmed": true,
  "action_id": "act_8f3a9e1b2c4d",
  "service": "ec2",
  "tool_name": "create_instance",
  "parameters": {
    "instance_name": "web-demo",
    "os_type": "ubuntu",
    "instance_type": "t3.micro"
  },
  "region": "ap-southeast-1"
}
```

Mẫu JSON Response minh họa khi Dispatcher Lambda và module EC2 Service thực thi thành công trong khu vực Singapore:

```
{
  "status": "SUCCESS",
  "type": "action_executed",
  "service": "ec2",
  "tool_name": "create_instance",
  "message": "Đã gửi yêu cầu tạo máy chủ web-demo.",
  "data": {
    "instance_id": "i-015aad95042137744",
    "instance_name": "web-demo",
```

```
    "state": "pending",
    "instance_type": "t3.micro",
    "image_id": "ami-0af9110f0ec0820e2"
  }
}
```

3. Bảng giải thích chi tiết các trường dữ liệu:

Bảng 7. Mô tả các trường trong Request Body của Endpoint `POST /confirm`

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả ý nghĩa và nguồn gốc dữ liệu
`confirmed`	Boolean	Có	Cờ xác nhận mang giá trị `true`, khẳng định người dùng đã trực tiếp phê duyệt hành động.
`action_id`	String	Có	Mã hành động tương ứng đã tạo ở bước `/chat`, dùng để đối soát trạng thái trong bảng `PendingAction`.
`service`	String	Có	Dịch vụ AWS đích cần điều phối thực thi (`ec2`, `s3`, `lambda`, `iam`).
`tool_name`	String	Có	Tên hành động kỹ thuật cần thực thi (`create_instance`, v.v.).
`parameters`	Object	Có	Tập tham số kỹ thuật đã qua rà soát của người quản trị.
`region`	String	Không	Khu vực thực thi lệnh trên đám mây AWS (mặc định: `ap-southeast-1`).

Bảng 8. Mô tả các trường trong Response Body của Endpoint `POST /confirm`

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả ý nghĩa và nguồn gốc dữ liệu
`status`	String	Có	Kết quả xử lý hạ tầng tổng thể (`SUCCESS` hoặc `ERROR`).
`type`	String	Có	Phân loại giao diện (`action_executed`) để Web Client cập nhật trạng thái thẻ hiển thị sang hoàn tất.
`service`	String	Có	Tên dịch vụ AWS đã thực thi hoàn tất (`ec2`).
`tool_name`	String	Có	Tên công cụ kỹ thuật đã chạy (`create_instance`).
`message`	String	Có	Thông báo tổng kết kỹ thuật bằng tiếng Việt kèm mã ID và trạng thái máy ảo.
`data`	Object	Có	Đối tượng chứa dữ liệu chi tiết trả về từ Boto3 SDK gồm: `instance_id`, `instance_name`, `state` (trạng thái `pending` non-blocking), `instance_type` và mã `image_id` chính thức của hệ điều hành.

3. KẾT QUẢ THỰC HIỆN

Chương này trình bày quy trình triển khai hoàn chỉnh hệ thống SatiBot lên môi trường điện toán đám mây AWS tại tên miền https://satibot.id.vn/ cùng các kết quả kiểm thử, nghiệm thu cho quy trình CI/CD, bảo mật biên và chuyển đổi dự phòng, các kịch bản quản trị dịch vụ AWS qua giao diện ChatOps và đánh giá hiệu năng vận hành thực tế.

3.1 Các bước triển khai hệ thống thực tế (Deployment Steps)

Hệ thống được mô hình hóa hoàn toàn theo Infrastructure as Code (IaC) bằng AWS CDK và triển khai theo thứ tự phụ thuộc Multi-Region giữa region chính Singapore (`ap-southeast-1`) và region dự phòng Bắc Virginia (`us-east-1`):

1. **Khởi tạo tầng dữ liệu và phân quyền:** Thực thi `cdk bootstrap` trên cả hai region; triển khai 5 bảng DynamoDB (user, AWS connections, session state, pending action, action history) và các IAM Role theo nguyên tắc Least Privilege tách riêng cho từng hàm Lambda và dịch vụ CodeDeploy.
2. **Thiết lập bảo vệ biên và trang fallback Multi-Region:** Triển khai Web ACL của AWS WAF, cấp chứng chỉ SSL/TLS bao phủ tên miền `satibot.id.vn` và tạo S3 bucket riêng tư chứa trang fallback tĩnh (Maintenance Page) tại `us-east-1`.
3. **Đóng gói và triển khai Backend Serverless:** Chạy Unit Test, đóng gói 5 hàm Lambda nghiệp vụ (`Chatbot`, `Dispatcher`, `Formatter`, `Connector`, `History`) cùng Workflow Step Functions, cấu hình alias `live` gắn CodeDeploy Canary và 5 CloudWatch Alarm.
4. **Kết nối lớp xác thực và giao tiếp real-time:** Triển khai Amazon Cognito (OAuth2/PKCE), API Gateway REST API tích hợp trực tiếp tới alias `live`, và AWS AppSync GraphQL/WebSocket để đẩy kết quả từ backend về trình duyệt ngay khi hoàn tất.
5. **Deploy giao diện, gắn tên miền và kích hoạt CI/CD:** Đưa ứng dụng React lên S3 qua CloudFront CDN gắn Origin Group dự phòng; cấu hình Route 53 DNS; đồng thời nối GitHub App tới AWS CodePipeline để tự động hóa toàn bộ quy trình CI/CD.

Bảng 9 tổng hợp các nhóm dịch vụ thuộc 12 CloudFormation stack đã triển khai.

Bảng 9. Kiến trúc triển khai thực tế của hệ thống SatiBot trên hai region AWS

Tầng kiến trúc	Dịch vụ AWS (Region)	Vai trò trong hệ thống vận hành
Giao diện & Bảo vệ biên	S3, CloudFront, Route 53, ACM, AWS WAF (`ap-southeast-1` & `us-east-1`)	Phân phối ứng dụng qua HTTPS, định tuyến DNS, chặn XSS/SQLi/Rate-limit và trả về trang fallback khi origin chính lỗi.

Xác thực & Kết nối API	Amazon Cognito, API Gateway, AWS AppSync (`ap-southeast-1`)	Xác thực người dùng bằng JWT, tiếp nhận request hội thoại/xác nhận và đẩy kết quả real-time về trình duyệt qua WebSocket.
Xử lý Serverless & Điều phối	5 Lambda nghiệp vụ, Lambda `HealthCheck`, Step Functions (`ap-southeast-1`)	Phân tích ý định qua Gemini API, thực thi thao tác trên tài khoản AWS đích bằng STS `AssumeRole` và định dạng phản hồi.
Dữ liệu & Bảo mật nội bộ	5 bảng DynamoDB, IAM, Secrets Manager (`ap-southeast-1`)	Lưu session, lịch sử thao tác có audit log, cách ly IAM policy cho từng hàm và bảo vệ khóa Gemini API.
CI/CD & Giám sát	GitHub Actions, CodeConnections, CodeBuild, CodePipeline, CodeDeploy, CloudWatch	Chạy test tự động, `cdk synth`, Manual Approval, triển khai Canary và Auto Rollback khi có lỗi.

3.2 Nghiệm thu quy trình CI/CD và triển khai Canary không gián đoạn

Quy trình phát hành liên tục của SatiBot được kiểm chứng hoàn chỉnh từ lúc push code lên nhánh `main` trên GitHub đến khi cập nhật thành công toàn bộ cụm backend đang chạy tại https://satibot.id.vn/ mà không gây gián đoạn người dùng.

Song song với 2 workflow trên GitHub Actions (*Backend CI* và *Deploy Infrastructure* đều hoàn tất thành công), toàn bộ **518 Backend/CDK test** (`pytest`) cùng **20 Frontend Contract test** (`node --test`) đều pass **100%** (0 fail). Trên AWS CodePipeline, CI/CD pipeline gồm 4 stage nối tiếp đã hoàn tất trọn vẹn **4/4 Stage (`Succeeded`)** như Bảng 10.

Bảng 10. Kết quả nghiệm thu CI/CD pipeline và triển khai Canary trên AWS

Stage / Kịch bản	**Kết quả**	**Minh chứng thực tế trên hệ thống**
1. `Source` (GitHub → AWS)	**Đạt**	AWS CodeConnections tự động phát hiện commit mới trên nhánh `main` và tải source code vào pipeline chỉ trong vài giây.
2. `BuildTestSynth` (CodeBuild CI)	**Đạt**	Chạy lại toàn bộ `pytest`, kiểm tra 6 gói ZIP Lambda, chạy `cdk synth` cho 5 app CDK và sinh file cấu hình deploy.
3. `ManualApproval` (Cổng duyệt)	**Đạt**	Giữ pipeline ở trạng thái chờ an toàn trước khi thay đổi hạ tầng production; chỉ tiếp tục sau khi người vận hành bấm Approve trên AWS Console.
4. `DeployBackend` (CloudFormation & CodeDeploy)	**Đạt**	Publish CDK assets lên S3, tạo ChangeSet có kiểm tra tham số tích hợp, cập nhật stack backend về trạng thái `UPDATE_COMPLETE` và giữ endpoint `/api/health` trả HTTP 200 xuyên suốt quá trình.
Kiểm nghiệm phát hành Canary không gián đoạn	**Đạt (5/5 hàm)**	CodeDeploy dịch chuyển 10% traffic sang version mới trong 5 phút dưới sự giám sát của 5 CloudWatch Alarm (trạng thái `OK`), sau đó chuyển toàn bộ 100% traffic.
Kiểm nghiệm Auto Rollback khi dừng triển khai	**Đạt**	Khi chủ động dừng một đợt deploy bất thường, CodeDeploy tự động rollback về version ổn định trước đó mà không làm gián đoạn dịch vụ.

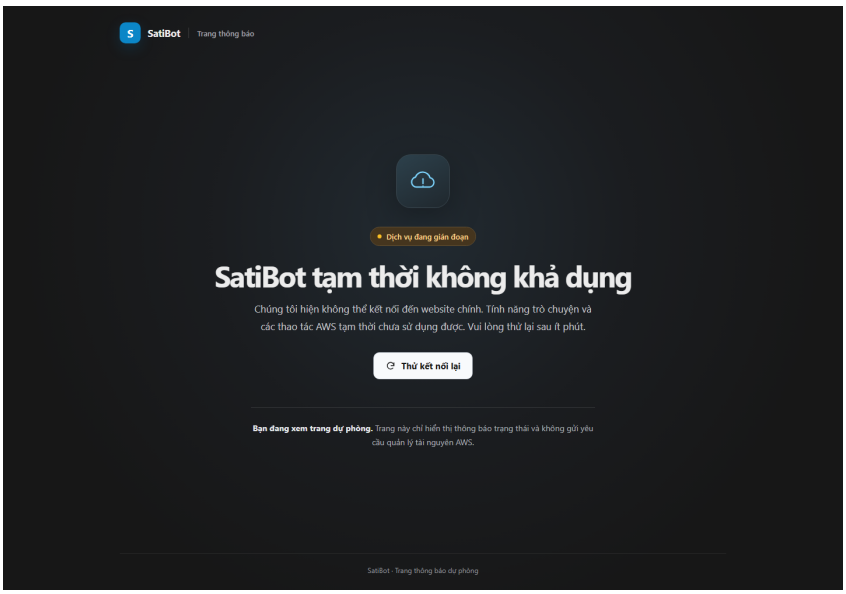
3.3 Nghiệm thu Route 53 DNS, bảo mật AWS WAF và Origin Failover

Nhóm tiến hành kiểm tra đồng bộ cấu hình Route 53 DNS, chứng chỉ ACM (`us-east-1`), bộ rule bảo mật AWS WAF tại CloudFront và cơ chế CloudFront Origin Failover khi origin chính không phản hồi. Bảng 11 tóm tắt kết quả kiểm thử thực tế trên các điểm truy cập công khai.

Bảng 11. Kết quả kiểm thử định tuyến tên miền, bảo mật biên WAF và Origin Failover

Kịch bản kiểm thử	**Cơ chế xử lý**	**Kết quả thực tế**
Truy cập trang web chính (`satibot.id.vn`)	Route 53 Alias → CloudFront CDN → S3	Trả **HTTP 200**; đính kèm các header bảo mật và kiểm soát cache (`HSTS`, `X-Frame-Options`, `X-Content-Type-Options`, `Cache-Control: no-store`).
Kiểm tra Health Check API (`api.satibot.id.vn`)	Route 53 Failover (Primary) → API Gateway	Phân giải qua bản ghi Primary Failover tại endpoint `/api/health` gắn Route 53 Health Check (*Healthy*), trả **HTTP 200** (`API_READY`).
Gọi API thiếu JWT token (`/api/history`)	Amazon Cognito JWT Authorizer	Từ chối bằng **HTTP 401** (*Unauthorized*), chặn request chưa xác thực đi vào Lambda.
Kiểm thử tấn công XSS qua query string	AWS WAF (`CommonRuleSet`)	Chặn tại CloudFront edge với **HTTP 403**; hệ thống WAF ghi nhận sự kiện `BLOCK`.
Kiểm thử tấn công SQL Injection (SQLi)	AWS WAF (`SQLiRuleSet`)	Chặn tại edge với **HTTP 403**, chặn payload thử nghiệm trước khi chuyển tới backend qua CloudFront.
Kiểm thử Rate Limiting (100 req / 5 phút / IP)	AWS WAF Rate-based Rule	Tự động chặn luồng request vượt ngưỡng với **HTTP 403**, chống spam và quá tải hệ thống.
Mô phỏng Origin chính lỗi `HTTP 504`	CloudFront Origin Group Failover (→ S3 `us-east-1`)	CloudFront thử kết nối 1 lần; khi nhận mã `504`, tự động failover sang bucket S3 dự phòng qua OAC và trả **HTTP 200** hiển thị trang fallback (Hình 8).

Trong phạm vi bảo vệ hệ thống production tại https://satibot.id.vn/, cơ chế Route 53 Failover đã được gắn trực tiếp cho endpoint API kèm Health Check tự động; kịch bản giả lập lỗi `504` để kiểm chứng chuyển hướng sang trang fallback tại `us-east-1` được thực hiện trên một CloudFront distribution thử nghiệm độc lập trỏ chung bucket S3 dự phòng nhằm tránh gián đoạn website chính của người dùng.



Hình 8. Trang fallback tĩnh phục vụ tự động từ S3 tại `us-east-1` khi origin chính gặp sự cố 504

3.4 Nghiệm thu các kịch bản quản trị AWS và đánh giá hiệu năng

3.4.1 Nghiệm thu sáu kịch bản Amazon S3

Bảng 12. Kết quả nghiệm thu các kịch bản S3 trên website bản thử

STT	Kịch bản	Kết quả quan sát
1	Tạo bucket an toàn	**Đạt.**Tạo bucket private thành công tại Singapore, bật đủ Block Public Access.
2	Quét/khóa public access	**Đạt.** Quét 5 bucket, không phát hiện bucket rủi ro; bật lại đủ bốn cờ chặn cho bucket thử thành công.
3	Xóa có xác nhận	**Đạt.** Xem nội dung trước khi xóa; xác nhận bằng đúng tên bucket; xóa bucket cùng 2 mục bên trong.
4	Lifecycle	**Đạt.** Tạo rule cho prefix `demo/`, chuyển sang `STANDARD_IA` sau 30 ngày.
5	Presigned download	**Đạt.** Link thời hạn 15 phút tải được đúng tệp mẫu, phản hồi HTTP 200.
6	Dung lượng và chi phí	**Đạt.** Thống kê 2 object, 1.000.038 byte (xấp xỉ 0,001 GB), lớp `STANDARD`; ước tính 0,000025 USD/tháng.

Chi phí trong bảng là **ước tính** theo đơn giá tham chiếu, không phải hóa đơn AWS; thao tác lifecycle chỉ xác minh cấu hình rule, chưa chờ đủ 30 ngày để quan sát dịch chuyển storage class.

3.4.2 Nghiệm thu các kịch bản Amazon EC2

3.4.3 Nghiệm thu các kịch bản AWS IAM

3.4.4 Nghiệm thu kịch bản quản trị AWS Lambda qua giao diện ChatOps

Thông qua giao diện chat tiếng Việt trên https://satibot.id.vn/ kết nối bằng vai trò liên kết STS `AssumeRole` sang tài khoản AWS đích, nhóm đã kiểm thử toàn bộ **12/12 công cụ quản trị Lambda** cùng các cơ chế Guardrail an toàn. Toàn bộ kết quả hiển thị trên khung chat khớp hoàn toàn với trạng thái tài nguyên trên AWS Console và được ghi nhận đầy đủ trong bảng `ActionHistory` (Bảng 13).

Bảng 13. Tổng hợp kết quả nghiệm thu các kịch bản và Guardrail quản trị Lambda

Kịch bản & Guardrail an toàn	**Kết quả thực hiện và đối chiếu thực tế**
1. Audit danh sách, config và source code Lambda	**Đạt.** Liệt kê chính xác toàn bộ các hàm Lambda trong region; đọc đúng `runtime`, `RAM`, `timeout` và tải lại chính xác file mã nguồn Python đang chạy trên AWS.
2. Chẩn đoán sức khỏe Lambda (*Health Audit*)	**Đạt (100/100 điểm).** Tự động phát hiện các hàm cấu hình tài nguyên thấp; sau khi chuẩn hóa RAM $\geq$ 256 MB và timeout $\geq$ 15 s, cả 7 hàm đều đạt trạng thái `HEALTHY` theo bộ tiêu chí kiểm tra cấu hình của công cụ.
3. Tạo hàm mới & *Conflict Guardrail*	**Đạt.** Tạo thành công hàm Python 3.12 mới với mã nguồn do người dùng/LLM cung cấp. Khi gặp tên hàm đã tồn tại, hệ thống dừng lại yêu cầu chọn hướng xử lý (update, đổi tên, cancel) thay vì tự ý ghi đè.
4. *AST Validation Guardrail* trước khi deploy	**Đạt.** Tự động chặn sớm các lỗi cú pháp, các lời gọi bị cấm theo quy tắc của dự án (`eval`/`exec`) và việc dùng nhầm các định danh theo cú pháp JSON (`true`/`false`/`null` thay cho `True`/`False`/`None`).
5. Update code & Invoke hàm trực tiếp qua Chat	**Đạt.** Đóng gói, cập nhật phiên bản code mới và invoke trực tiếp hàm với test payload JSON, trả về HTTP 200 cùng output và execution duration.
6. Hot Config tài nguyên (RAM / Timeout)	**Đạt.** Cập nhật cấu hình bộ nhớ và timeout ngay trên khung chat sau bước xác nhận; cấu hình cập nhật thành công và khớp hoàn toàn trên AWS Console.
7. Tạo Function URL & *CORS Wildcard Guardrail*	**Đạt.** Từ chối CORS origin `*` theo quy tắc của dự án; chỉ cấp phát HTTPS Endpoint công khai khi truyền origin hợp lệ (https://satibot.id.vn), gọi thử trả HTTP 200. CORS giới hạn truy cập khác origin từ trình duyệt, không thay thế cơ chế xác thực.
8. Cấu hình *Resilient Service* (Alias, Async Retry)	**Đạt.** Tự động publish Version, gán Alias `live`, bật Async Retry, đồng thời thực hiện Graceful Degradation kèm warning khi tài khoản chạm giới hạn concurrency.
9. *Emergency Kill-Switch* (Throttle)	**Đạt.** Đặt `Reserved Concurrency = 0` ngăn phát sinh lượt thực thi mới; sau đó gỡ bỏ giới hạn để khôi phục hoạt động bình thường.
10. *Core Deny Guardrail* & Delete hàm thử nghiệm	**Đạt.** Lệnh yêu cầu xóa hàm core của chính SatiBot bị từ chối ở tầng IAM (`AccessDenied`); các hàm thử nghiệm được xóa và xác minh không còn tồn tại.

3.4.5 Đánh giá hiệu năng vận hành thực tế và bài học tối ưu kỹ thuật

Qua theo dõi CloudWatch Logs trên tổng số **1.125 lượt invoke** (gồm **144 Cold Start** và **981 Warm Start**), cùng 144 lần chạy Step Functions workflow, hệ thống vận hành ổn định và không ghi nhận bất kỳ Unhandled Exception / Traceback nào (Bảng 14).

Bảng 14. Hiệu năng vận hành thực tế của 6 hàm AWS Lambda trong hệ thống SatiBot

Hàm Lambda	RAM / Timeout	Invoke & Cold Start	Init Duration, Latency & Đặc điểm vận hành
`Chatbot`	256 MB / 60 s	273 lượt (25 cold)	Init 128–181 ms; latency TB $\sim$ 3,1 s (chủ yếu do chờ API Gemini LLM); Peak RAM 107 MB.
`Dispatcher`	512 MB / 60 s	138 lượt (19 cold)	Init 272–394 ms; latency TB $\sim$ 2,1 s (gồm lấy credential tạm qua STS và gọi Boto3 đa tài khoản); Peak RAM 255 MB.
`Formatter`	256 MB / 30 s	132 lượt (17 cold)	Init 81–112 ms; latency TB $\sim$ 3,1 s (chờ Gemini tổng hợp và phát AppSync GraphQL); Peak RAM 96 MB.
`Connector`	256 MB / 30 s	94 lượt (32 cold)	Init 297–398 ms; latency TB $\sim$ 0,5 s (kiểm tra kết nối STS và ghi cấu hình DynamoDB); Peak RAM 93 MB.
`History`	256 MB / 30 s	403 lượt (18 cold)	Init 254–372 ms; Warm **2–9 ms** (TB 76 ms) khi query DynamoDB; Peak RAM 92 MB.
`HealthCheck`	256 MB / 15 s	85 lượt (33 cold)	Init 66–101 ms; Warm **1–2 ms** (TB 7,6 ms) phục vụ Route 53 Health Check; Peak RAM 33 MB.

Từ số liệu đo đạc thực tế, nhóm rút ra ba điểm cải tiến kỹ thuật cốt lõi đã được kiểm chứng và đưa vào hệ thống vận hành:

1. **Phân tách rõ Latency Bottleneck:** Tầng serverless nội bộ phản hồi rất nhanh ở trạng thái Warm (**2–9 ms** ở `History` và **1–2 ms** ở `HealthCheck`); Chatbot và Formatter có latency trung bình khoảng 3,1 giây mỗi hàm, chủ yếu do gọi Gemini LLM API, nên kiến trúc bất đồng bộ Step Functions kết hợp AppSync WebSocket giúp UI không bị block trong lúc chờ xử lý.
2. **Giới hạn mức tăng bộ nhớ của `Dispatcher` cache:** Phân tích CloudWatch Logs phát hiện bộ nhớ sử dụng tăng dần khi lưu các session/client phục vụ STS `AssumeRole` tới sát ngưỡng 255/256 MB sau khoảng 10 lần gọi liên tiếp. Nhóm đã giới hạn kích thước cache tối đa 8 session, tái sử dụng `boto3.client` dùng chung tài nguyên và chuẩn hóa nâng RAM của `Dispatcher` lên **512 MB** qua CI/CD pipeline.
3. ***AST Guardrail* khắc phục lỗi sinh code của LLM:** Bổ sung các quy tắc kiểm tra trên AST trước khi đóng gói giúp phát hiện việc dùng nhầm các định danh theo cú pháp JSON (`true`/`false`/`null`) và gợi ý sửa về chuẩn Python (`True`/`False`/`None`), nâng độ ổn định khi tạo và cập nhật Lambda bằng ngôn ngữ tự nhiên.

3.5 Vai trò và đóng góp của các thành viên

Bảng 15. Phân công và kết quả đóng góp của Nhóm 10

Thành viên	Vai trò phụ trách và kết quả bàn giao
Cao Anh Minh **(Trưởng nhóm)**	Thiết kế kiến trúc tổng thể, phân quyền IAM, mô hình 5 bảng DynamoDB, điều phối tích hợp đa dịch vụ (Step Functions, AppSync, nền tảng CDK Data/IAM) và tổng hợp báo cáo.
Trịnh Thiên Quang	Phát triển cụm 5 Lambda backend, tích hợp Gemini API, thiết lập CI/CD Pipeline (*CodePipeline/CodeDeploy Canary*), tối ưu memory cache và nghiệm thu 12/12 công cụ quản trị AWS Lambda.
Phạm Đức Hùng	Xây dựng module quản trị và nghiệm thu 6 kịch bản Amazon S3; phát triển giao diện React, kết nối frontend AppSync, cấu hình AWS WAF và trang fallback tại `us-east-1`.
Nguyễn Đức Huy	Xây dựng module quản trị Amazon EC2, lớp xác thực Amazon Cognito / API Gateway, cấu hình tên miền `satibot.id.vn`, chứng chỉ ACM và định tuyến Route 53 Health Check / Failover.

3.6 Liên kết sản phẩm, tài liệu bổ trợ và kết luận

- **Ứng dụng đã triển khai:** https://satibot.id.vn/ (API Health Check: https://api.satibot.id.vn/api/health).
- **Mã nguồn GitHub:** https://github.com/minhcaoj/SitaBot.
- **Slide báo cáo cuối kỳ:** *[Bổ sung đường dẫn slide khi nộp bài]*.
- **Video trình bày và demo hệ thống:** *[Bổ sung đường dẫn video sau khi quay và tải lên]*.
- **Tài liệu bổ trợ & Nhật ký công việc:** Báo cáo Progress 1, Progress 2, bộ test tự động (`tests/`, `frontend/`), dữ liệu đối chiếu WAF (`evidence/`) và lịch sử commit/Pull Request trên GitHub (biên bản họp nhóm đính kèm khi nộp).

Kết luận. Nhóm đã triển khai và đưa vào vận hành hoàn chỉnh hệ thống SatiBot tại https://satibot.id.vn/ với hạ tầng khai báo bằng AWS CDK, backend tại region chính và trang thông báo dự phòng tại region thứ hai, quy trình CI/CD kết hợp CodeDeploy Canary từ GitHub lên AWS có bước phê duyệt thủ công, lớp bảo mật biên WAF và Origin Failover hoạt động ổn định, cùng phân hệ quản trị ChatOps an toàn qua AWS Lambda và Amazon S3.